

Defense speaking copy

1. Title

0:21

Good afternoon. I'm Sakif Khan. This is my thesis defense on Agentic Graph Reasoning, which is knowledge graph navigation with verification before the answer is emitted. My supervisor is Dr. Sadia Sharmin.

2. The problem

0:32

Language models answer factual questions fluently, and just as fluently when they do not hold the fact. Fluency and factuality come out of the same mechanism, so nothing in the wording separates them. A system that is confidently wrong is worse than one that says it does not know.

3. What a hallucination is

0:39

First, the word. A hallucination is fluent, confident, and backed by no source the model can point to.

Two kinds, separated by what it takes to catch them. Factually wrong: you need the true answer already, which is what we are trying to produce. Unsupported: you need only what was retrieved, and we have that. I claim the second.

4. Where it comes from, and what we can change

0:35

It is not a bug that escaped testing. It comes from how these models are built, so the response has to be architectural.

Three origins. The training data, the model itself, and the prompt. Only the prompt is ours to change. So control what goes in, and check what comes out against it.

5. Knowledge graphs, and why multi-hop is hard

0:49

A knowledge graph stores facts as triples, so every fact has an address and a claim can be checked by looking for an edge. A question that chains two facts is multi-hop, and a wrong first hop is never revisited.

One complication. Freebase stores a fact with more than two participants as an unnamed node, and nearly two thirds of this graph's nodes are that kind. So nothing here is called "plays". Hold onto that.

6. What guided beam search is

0:46

You hold a set of current nodes, the frontier. List every edge out of it, score each, keep the best few, expand those, and repeat to a depth cap. The number you keep is the beam width.

Guided means the score is a judgement, here a model call. So the width is the cost, and it multiplies with every hop. And whatever falls outside the beam is gone for good.

7. Where existing approaches stop

0:41

Four families, and each stops somewhere specific. The first three fix their evidence before reasoning begins. Agentic navigation interleaves the two, which is the right move, and it already names only what it traversed.

So its problem is precision. A real, traversed entity can still be the wrong answer, and none of the four checks what the answer asserts before it goes out.

8. What everyone else does

0:43

Four prior systems, and the column that matters is the last one. They differ in how they explore, some decompose, and Plan-on-Graph backtracks. None checks its answer's claims against what it retrieved, before answering. RoG comes closest, since its answers are grounded by construction, but nothing examines them. Post-hoc checkers such as Chain-of-Verification check after generation.

Their published accuracies are higher than mine and not comparable.

9. Research questions

0:34

Three questions. RQ1: does agentic navigation improve multi-hop accuracy, and does the advantage grow with hop count? RQ2: what does pre-generation verification contribute beyond graph navigation? RQ3: which components contribute what, at what token cost?

On the right are the six contributions the thesis claims. I return to them at the end.

10. AGR: An explicit state machine

0:53

AGR is an explicit state machine, not a prompt loop. Six nodes. The planner writes sub-objectives. The explorer scores candidate edges and keeps the best three per anchor. The evaluator decides whether an objective is met. The backtracker returns to the best-scoring earlier frontier and bans the edges that failed. The verifier checks the draft's claims, and the answerer emits what survived.

Three cycles, the three arrows back to the explorer, and all three are bounded by budgets checked in code.

11. Constrained tools, not free-form queries

0:33

One agent, six nodes, and only three touch the graph. The planner resolves entity mentions, the explorer asks for relations and neighbours, and the verifier checks adjacency. Four operations, never Cypher. Every call is logged, so the traversal is a record, and that record is what the verification layer checks against.

12. The Structural Verification Layer

0:37

This is the part I set out to contribute. Before anything is emitted, the draft is split into atomic claims, each checked against the graph, the traversed triples first. What cannot be grounded is re-explored or dropped. So where it cannot ground a claim, the system hedges rather than asserts. Measured, it buys auditability rather than accuracy.

13. One claim, three routes

0:45

This is the inside of the verifier. One model call drafts the answer and splits it into claims. Each claim leaves at the first test that settles it, left to right. Traversed adjacency comes first, and it is the only route that adds evidence. Then verify connection, on the full graph. Only the third route, entailment, spends a model call. With no unsupported claim the answer goes out grounded.

14. What *structural* means, and what it does not

0:58

Now the bound. The graph decides. The model is consulted only where the graph is silent, and it never overrides the graph.

Structural does not mean it can read the relation. The first two routes test adjacency, in either direction, so a mother claim survives on a child edge. That is the mediator problem as a limitation.

“Its evidence” is narrower too. Only the walked-graph check attaches triples, and the log keeps a count, not the triples. And a claim can be true and still be the wrong answer.

15. One question, end to end

0:44

One real question, from the committed records. “Who plays Dwight in The Office.” The dataset gives the mention The Office, and the planner resolves it and splits the question. The explorer keeps three relations and the evaluator resolves Dwight Schrute. Second hop, and it resolves Rainn Wilson and answers. The verifier finds two claims, both supported. Six model calls, depth two, and at most eighteen supporting triples.

16. The environment and the question sets

0:48

The environment is the union of RoG's per-question subgraphs, 2.6 million entities and 8.3 million triples. Each subgraph was cut to contain its own answer, so this is friendlier than full Freebase.

400 questions each from WebQSP and ComplexWebQuestions, unmodified. Every question's gold answer is certified reachable, 97 percent and 99.2. That is the ceiling on every accuracy figure, for entity answers only. Dates, quantities and superlatives this graph cannot express at all.

17. Making the comparison fair

1:08

Five systems on one frozen backbone, under the same 25-call budget, on the same questions and graph. That is what lets me attribute differences to architecture rather than to model capacity. Think-on-Graph is my reimplementation, and the graph systems all start from the datasets' topic entities.

Two things are not equal, and I would rather say them than have them found. Think-on-Graph prunes from a narrower candidate set. That cuts against it, since a thinner set is cheaper, so it cannot explain the clipping, but it makes its score a lower bound. And AGR's drafting prompt carries grounding rules the baselines' plain prompt does not.

18. Main results

1:21

Here is the comparison.

AGR reaches 0.755 Hits@1 and 0.642 F1 on WebQSP, and 0.522 and 0.469 on ComplexWebQuestions, ahead of every baseline on both. All eight paired tests reject.

First, vector RAG on ComplexWebQuestions: 0.203, *below* the no-retrieval control at 0.307. One verbalised triple cannot contain a chain, so single-shot retrieval is worse there than not retrieving at all. GraphRAG sits beside it at 0.205, but its one-hop radius confounds the paradigm, so the claim rests on vector RAG.

On WebQSP the control even beats GraphRAG on raw hits, by guessing. It is right on 51.6 percent of what it asserts, GraphRAG on 76.8.

And cost: about a quarter more tokens than Think-on-Graph, at half the calls, and calls are what the budget meters.

19. AGR against RoG

1:06

RoG is the one published system measured on this same data. My graph is built from its subgraphs. And it is ahead on all four figures, outside my confidence intervals. It is not the scorer. Scored by RoG's own code, my numbers move by under a point.

Two things differ. RoG is fine-tuned on both benchmarks' training splits, and AGR trains on nothing, which is a difference in kind. And RoG searches each question's own subgraph, close to an oracle, where I search the union of all of them.

So this places AGR among published systems. It does not compare architectures.

20. RQ1: Does agentic navigation improve multi-hop factual accuracy?

0:56

The answer to RQ1 is yes, and the evidence is a shape.

ComplexWebQuestions, on the right. AGR goes 0.46, 0.55, 0.57 as questions get harder. It is the only system on that dataset that ends above where it started. Three of the other four decay monotonically, and Think-on-Graph falls and partially recovers, still 0.08 below its own one-hop score. It does beat AGR at one hop, 0.53 against 0.46, where decomposition is not needed.

The left panel is dashed because its three-hop stratum is four questions.

21. The caveat I want to raise myself

1:01

I want to raise this before you do.

Split the questions by whether Think-on-Graph finished inside the shared 25-call cap. On the ones it finishes, it is ahead of AGR, 0.852 against 0.788 on WebQSP and 0.629 against 0.607 on CWQ.

The entire margin comes from the questions it cannot finish, 29 percent of WebQSP and 44 percent of CWQ. Beam search pays width times depth in calls, and AGR's plan does not multiply.

So the claim is not that AGR reasons better per step. It is that AGR finishes inside a fixed budget.

22. RQ2: What does pre-generation verification contribute beyond graph navigation?

0:50

RQ2, in its narrowest form. Does anything ungrounded get asserted?

Across both datasets, AGR asserts 1,709 entities and zero are ungrounded. The parametric control asserts 1,001 and 22.1 percent are ungrounded.

That looks like a headline for the verification layer. It isn't. Think-on-Graph also reaches zero, with no verification layer at all. So this is a property of navigating a graph. And a stricter semantic check still fails a third to a half of every system's answers.

23. RQ2: What verification does *not* do

0:50

Removing the layer changes accuracy by nothing I can detect, p equals 1.0 on both datasets. On test it changed two answers, one each way.

Nor does it explain the precision lead. On CWQ, AGR is right on 67.9 percent of what it asserts, Think-on-Graph on 56.1, and removing the layer barely moves AGR's. The lead belongs to the architecture, and perhaps the prompt.

And its evidence ends with the run. The log keeps a count.

24. RQ2: So what does it do?

0:41

What it does is withhold what it cannot ground. Without it, CWQ hedging falls from 23.2 to 20.2 percent. Those six answers were all wrong. On WebQSP it withheld one, and that one was right.

Its clearest catch was a draft claiming Marlon Brando was in Joy. The verifier rejected it. Not right more often, but right for a reason it can show.

25. Does it just refuse more often?

0:31

The obvious objection, and the answer is no. AGR hedges least of the five on WebQSP, where it is also the most accurate. On CWQ it is within half a point of Think-on-Graph, and nearly nine points more accurate. A hedge scores zero, like a wrong answer.

26. RQ3: Which components contribute what, at what token cost?

0:24

RQ3: which components earn their cost. Four ablations, each a paired test against the full system on the same halves. Three change nothing I can detect: backtracking, the verifier, and the learned half of the scorer.

27. RQ3: One effect, and its sign is backwards

0:57

The one that does is the planner, and the sign is backwards. Removing it improves WebQSP by 0.083 F1 at p equals 0.006, and cuts tokens by 31 percent. On ComplexWebQuestions it trends the other way, and its two-hop stratum falls from 0.48 to 0.34.

The mechanism is context-stripping. Asked for the 33rd president who led during WW2, the planner split the question, WW2 was never used, and the run answered Woodrow Wilson. Undecomposed, it found Truman in three calls. So decomposition should be gated on question structure.

28. Every failure, read and labelled

0:20

Two hundred and fifty-six failures, read and labelled by hand. The largest category is relation selection, the agent taking the wrong edge, not inventing one. That shape is the finding.

29. The echo attractor

1:01

The one to name is sixth, with 13 cases: the echo attractor, a real, grounded entity one hop from the answer. Asked who plays Lex Luthor on Smallville, AGR named the actor who plays his father.

Any grounding check passes it. It is real, it was traversed, the triple exists. True, and wrong. That is the promise from slide 14.

Different systems fall into it together, so no evaluation treating them as independent can see it. Rescore whenever a majority agrees, and this becomes apparent correctness. The contribution is the mechanism, not the count.

30. Contributions

0:58

Six contributions, the six the thesis claims. The one I'd underline is stratum-dependent decomposition. The literature treats it as plainly beneficial, and it is not. The protocol fixed four decisions in advance, and I missed one: the judge reached a kappa of 0.6995 against 0.7.

The limitations. The most serious is that the verifier persists only what it rejects, so wrongful acceptance has no rate. Then one environment, one backbone, one run, and the narrower candidate set. And the title is broader than my claims, as the thesis says.

31. Thank you

0:05

Thank you. I'm happy to take questions.